

PROJECT REPORT
ON
HOSPITAL WEBSITE AUTOMATION USING PYTHON



CCL
A MINI RATNA COMPANY

Submitted by –

Name – Ankur Gupta

Reg. No. – 23BCE0236

College – Vellore Institute of Technology Vellore

ACKNOWLEDGEMENT

I take this opportunity to thank Central Coalfield Ltd. Ranchi for giving me the privilege of working on such an interesting topic concerned with employee.

I am extremely grateful to the people of **GANDHI NAGAR HOSPITAL** who helped in making this report and successfully conducting the project that was allotted by **Mr. Dev Bhargha** (system dept.) whose guidance has enriched my knowledge horizon and provided without which the project could not have been success.

I would like to thank many other employees of CCL who have contributed greatly to the success of the project.

Finally, I wish to thanks all who have directly or indirectly helped me.

Ankur Gupta

CERTIFICATE

This is to certify that the project titled **Hospital Website automation** is a record of the Bonafide work done by **Ankur Gupta (23BCE0236)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in **COMPUTER SCIENCE & ENGINEERING** of **Vellore Institute of Technology**, Vellore, Tamil Nadu, during the academic year 2025-2026.

Signature:

Mr. Dev Bhargha
(System Department)

TRAINING CONDUCTED AT CCL

SYSTEM DEPARTMENT,
GANDHI NAGAR HOSPITAL, RANCHI,
JHARKHAND
DURATION: 4 WEEKS

SUBMITTED BY THE STUDENT OF **VELLORE INSTITUTE
OF TECHNOLOGY, VELLORE, TAMIL NADU**



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

TABLE OF CONTENTS

CHAPTER 1: CENTRAL COALFIELD LIMITED

CHAPTER 2: ABSTRACT

**CHAPTER 3: INTRODUCTION TO PYTHON & SELENIUM
AUTOMATION**

CHAPTER 4: AUTOMATION SCRIPT (WEBSITE CODE)

**CHAPTER 5: INTRODUCTION TO THE HMS PORTAL &
DATABASE**

CHAPTER 6: PROBLEM STATEMENT & OBJECTIVES

CHAPTER 7: METHODOLOGY

CHAPTER 8: RESULT & SNAPSHOTS

CHAPTER 9: CONCLUSION

CHAPTER 10: REFERENCES

CENTRAL COALFIELD LIMITED

Central Coalfield limited is a category-1 Mini Ratna Company since October 2007. During 2009 - 2010, coal production of the company reached its highest - ever figure of 47.08 million tonnes, with net worth amounting to Rs.2644 crore against a paid-up capital of Rs.940 crore. Formed on 1st November 1975, CCL (formerly National coal Development corporation Ltd.) was one of the five subsidiaries of coal India Ltd. Which was the first holding company for coal in the country (CIL now has 8 subsidiaries).

Early History - Formation of NCDC (Pre - nationalization)

CCL had a proud past. As NCDC, it heralded the beginning of nationalization of coal mines in India. National coal Development Corporation Ltd. (NCDC) was set up in October, 1956 as Government of India. It was started with nucleus of 11 old state collieries (owned by the railways) having a total annual production of 2.9 million tons of coal. Until the formation of NCDC, coal mining in India was largely confined to Rani Ganj coal belt in West Bengal and the Jharia coalfields in Jharkhand, besides a few other areas in Jharkhand and part a part of Madhya Pradesh, Chhattisgarh and Orissa.

From its very beginning, NCDC addressed itself to the task of increasing coal production and developing new coal resource in the outlying areas, besides introducing modern and scientific techniques of coal mining.

In the second five-year plan (1956-1961) NCDC was called upon to increase its production from new collieries, to be opened mainly in area away from the already developed Rani Ganj and Jharia coalfields. Eight new collieries were opened during this period and the production increased to 8.05 million tonnes by the end of the second plan.

ABSTRACT

Hospitals such as Gandhi Nagar Hospital under CCL use an internal web portal (HMSSO - Health Management System, Single Sign-On) to manage patient registration, OPD (Out-Patient Department) records, and various MIS reports. Staff in the System Department regularly need to log into this portal, navigate through several nested menus, and export reports such as the OPD Register for record-keeping and analysis.

This process, when done manually every day, is repetitive and time-consuming. The objective of this project was to build a Python-based automation script using Selenium WebDriver that logs into the HMSSO portal, navigates through the MIS Report menu structure, and opens the OPD Register report automatically, reducing manual effort and the chance of navigation errors.

The script was built using Selenium's WebDriver API along with web driver-manager for automatic Chrome Driver management, and was designed to be resilient to page load delays through the use of explicit waits.

INTRODUCTION TO PYTHON & SELENIUM AUTOMATION

What is Web Automation?

Web automation refers to the use of software tools to perform repetitive tasks on a website — such as logging in, clicking through menus, filling forms, or extracting data — without manual human interaction. Instead of a person clicking through the same sequence of steps every day, a script performs those exact actions programmatically, consistently, and much faster. This is especially valuable in enterprise settings where staff repeatedly need to navigate multi-level MIS (Management Information System) portals to reach a specific report.

Why Selenium?

Selenium is one of the most widely adopted open-source frameworks for browser automation. Unlike simple HTTP-request-based scraping tools, Selenium controls an actual browser instance (in this project, Google Chrome), meaning it can handle:

- JavaScript-rendered content that only appears after page scripts execute
- Dynamic, expandable menus (like the nested "MIS Report → Report → OPD Register" structure used in this project)
- Login sessions, cookies, and authentication flows exactly as a real user would experience them
- Pages that require waiting for elements to load before interacting with them

Because Selenium drives a real browser, it works reliably even on internal portals that were never designed with automation or APIs in mind — which is the case with most legacy government/PSU web applications, including CCL's HMSSO portal.

Why Python?

Python was chosen as the scripting language for this project for several reasons:

1. **Readable syntax** — automation logic (open page → find field → type → click) maps almost directly to plain English, making the script easy to maintain and hand off to other staff.
2. **Mature Selenium bindings** — the selenium package on PyPI is actively maintained and provides a complete WebDriver API.
3. **webdriver-manager** — this companion library automatically detects the installed Chrome version and downloads the matching ChromeDriver binary, removing the need to manually manage driver versions across different machines in the System Department.
4. **Cross-platform** — the same script runs on Windows or Linux machines in the hospital's IT setup with no code changes.

Core Selenium Concepts Used in This Project

Concept	Purpose	Where Used
<code>webdriver.Chrome()</code>	Launches and controls a Chrome browser session	<code>get_driver()</code>
By locator strategies	Identify HTML elements to interact with (by ID, link text, etc.)	<code>login()</code> , <code>click_menu_item()</code>
<code>WebDriverWait</code> + <code>expected_conditions</code>	Pause execution only until a specific condition is true (element present, clickable, or stale), instead of a fixed time. <code>sleep()</code>	Throughout the script
<code>EC.presence_of_element_located</code>	Waits until an element exists in the DOM	<code>open_site()</code> , <code>login()</code>
<code>EC.element_to_be_clickable</code>	Waits until an element is both visible and enabled	<code>click_menu_item()</code>
<code>EC.staleness_of</code>	Confirms the old page has been replaced (i.e., login succeeded and a new page loaded)	<code>login()</code>
Service + <code>ChromeDriverManager</code>	Automatically provisions the correct browser driver	<code>get_driver()</code>

AUTOMATION SCRIPT (PYTHON CODE)

The script below (hmsso.py) automates three stages of interaction with the hospital MIS portal: (1) launching the browser and loading the site, (2) logging in using stored credentials, and (3) navigating the nested MIS Report > Report > OPD Register menu path> Download excel sheet of patient information . Login credentials shown here are masked for security; the actual script uses the hospital's assigned system credentials.

```
"""
Automation starter for hmsso.coalindia.in
Requires: pip install selenium webdriver-manager
"""
import os
import time
import glob
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
URL = "https://hmsso.coalindia.in"
DOWNLOAD_DIR = os.path.join(os.path.expanduser("~"), "Downloads", "HMSSO_Reports")
def get_driver(headless: bool = False):
    os.makedirs(DOWNLOAD_DIR, exist_ok=True)
    options = Options()
    if headless:
        options.add_argument("--headless=new")
    options.add_argument("--start-maximized")
    options.add_argument("--disable-notifications")
    prefs = {
        "download.default_directory": DOWNLOAD_DIR,
        "download.prompt_for_download": False,
        "download.directory_upgrade": True,
        "safebrowsing.enabled": True,
    }
    options.add_experimental_option("prefs", prefs)
    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service, options=options)
    return driver
def open_site(driver):
    driver.get(URL)
    WebDriverWait(driver, 15).until(
        EC.presence_of_element_located((By.TAG_NAME, "body"))
    )
    print("Page loaded:", driver.title)
def login(driver, username, password):
    user_field = WebDriverWait(driver, 15).until(
        EC.presence_of_element_located((By.ID, "txtUserName"))
    )
    user_field.clear()
```

```

user_field.send_keys(username)
pass_field = driver.find_element(By.ID, "txtPassword")
pass_field.clear()
pass_field.send_keys(password)
login_btn = driver.find_element(By.ID, "btnLogin")
login_btn.click()
WebDriverWait(driver, 15).until(EC.staleness_of(login_btn))
print("Logged in.")
def click_menu_item(driver, link_text, wait_time=10):
    """Click a menu/link by its visible text (expands submenus, etc)."""
    elem = WebDriverWait(driver, wait_time).until(
        EC.element_to_be_clickable((By.PARTIAL_LINK_TEXT, link_text))
    )
    elem.click()
    print(f"Clicked: {link_text}")
def navigate_to_opd_register(driver):
    click_menu_item(driver, "MIS Report")
    click_menu_item(driver, "Report")
    click_menu_item(driver, "OPD Register")
def download_excel_report(driver, wait_time=20, before_files=None):
    """
    Click the 'Export to Excel' control on the OPD Register report page
    and wait for the .xls/.xlsx file to finish downloading.
    """
    before_files = before_files or set()
    excel_button = None
    candidate_locators = [
        (By.PARTIAL_LINK_TEXT, "Excel"),
        (By.XPATH, "//a[contains(@title, 'Excel') or contains(@alt, 'Excel')]"),
        (By.XPATH, "//img[contains(@title, 'Excel') or contains(@alt, 'Excel')]"),
        (By.XPATH, "//*[contains(@id, 'Excel')]"),
    ]
    for by, value in candidate_locators:
        try:
            excel_button = WebDriverWait(driver, 5).until(
                EC.element_to_be_clickable((by, value))
            )
            if excel_button:
                break
        except Exception:
            continue
    if not excel_button:
        raise RuntimeError(
            "Could not locate the Excel export control automatically. "
            "Inspect the OPD Register page and update download_excel_report()."
        )
    excel_button.click()
    print("Clicked Excel export control, waiting for download...")
    deadline = time.time() + wait_time
    downloaded_file = None
    while time.time() < deadline:
        current_files = set(glob.glob(os.path.join(DOWNLOAD_DIR, "*")))
        new_files = current_files - before_files
        finished = [f for f in new_files if not f.endswith(".crdownload")]
        if finished:
            downloaded_file = finished[0]
            break
        time.sleep(1)
    if downloaded_file:

```

```
        print("Downloaded file:", downloaded_file)
    else:
        print("Timed out waiting for the Excel file to download.")
    return downloaded_file
def main():
    driver = get_driver(headless=False)
    try:
        open_site(driver)
        login(driver, "GNHADMIN", "*****")
        navigate_to_opd_register(driver)
        before_files = set(glob.glob(os.path.join(DOWNLOAD_DIR, "*")))
        download_excel_report(driver, wait_time=20, before_files=before_files)
        input("Press Enter to close browser...")
    finally:
        driver.quit()
if __name__ == "__main__":
    main()
```

INTRODUCTION TO THE HMS PORTAL & DATABASE

What is HMSSO?

HMSSO (Health Management System - Single Sign-On) is the internal web-based portal used across Central Coalfield Limited's network of hospitals, including Gandhi Nagar Hospital in Ranchi. It serves as the digital backbone for hospital administration, covering patient registration, OPD (Out-Patient Department) visit logging, MIS reporting, and inter-departmental record access, all accessible through a single authenticated login, hence 'Single Sign-On'.

Role of the Portal in Hospital Operations

Rather than each department, such as the registration desk, pharmacy, records room, or administration, keeping separate paper or spreadsheet records, HMSSO centralizes this information. Patient visit data entered at registration is immediately available to doctors, pharmacy, and billing. Administrative staff in the System Department can generate consolidated MIS reports, such as the OPD Register, that summarize activity across the hospital for a given period. Data entered once is reused across multiple reports, reducing duplicate data entry.

Backend Database Context

Although this project did not require direct database access, it is useful context: every report the portal displays, including the OPD Register, is generated by querying a backend relational database (typically SQL Server or similar, common in these government MIS deployments) that stores patient and visit records. When a staff member navigates to 'OPD Register' through the web UI, the portal server runs a database query behind the scenes and renders the results as an HTML report.

This matters for automation because the portal exposes no public API; the only way to reach this data programmatically is by driving the web front-end itself, which is exactly what Selenium does. The HTML structure of the login form and menu, including element IDs like txtUserName, txtPassword, btnLogin, and link-text-based menu items like 'MIS Report' and 'OPD Register', had to be inspected directly from the live page using the browser's Inspect Element tool, since no API documentation exists for this internal system.

Menu Navigation Structure

The OPD Register report sits three levels deep in the portal's navigation: MIS Report (top-level menu, expands on click), followed by Report (sub-menu, expands on click), followed by OPD Register (the final report page). Each level must be expanded and clicked in sequence before the next becomes visible or clickable, which is why `click_menu_item()` was written as a reusable, generic function. It can locate and click any menu item by its visible link text, and is called three times in sequence inside `navigate_to_opd_register()` to walk this exact path.

PROBLEM STATEMENT & OBJECTIVES

Problem Statement: Staff at Gandhi Nagar Hospital's System Department manually log into the HMSSO portal, navigate through multiple nested menus, open the OPD Register report, and then manually click through to export it as an Excel sheet every time they need this data. This manual process is repetitive, slow, and prone to human error, especially when performed daily.

Objectives:

1. Automate the login process to the HMSSO portal using stored credentials.
2. Automate navigation through the MIS Report -> Report -> OPD Register menu path.
3. Automate triggering the Excel export on the OPD Register page and confirm the file is fully downloaded before proceeding.
4. Use explicit waits so the script reliably handles page load, menu expansion, and file-download delays.
5. Build a reusable structure (get_driver, open_site, login, click_menu_item, download_excel_report) that can be extended to other reports on the same portal.

METHODOLOGY

1. Requirement Gathering: Identified the exact manual steps performed by System Department staff to reach the OPD Register report and export it to Excel.
2. Tool Selection: Chose Python with the selenium and webdriver-manager packages, since they allow quick scripting of browser interactions without manual driver installation.
3. Element Inspection: Used the browser's Inspect Element tool on the live login page and the OPD Register report toolbar to identify the actual HTML IDs and link text for the username field, password field, login button, and Excel export control, replacing placeholder selectors in the script.
4. Script Development: Structured the script into small, single-purpose functions - `get_driver()` for browser setup (including configuring the default download folder), `open_site()` for loading the URL, `login()` for authentication, `click_menu_item()` for generic menu clicks, `navigate_to_opd_register()` to chain the required clicks, and `download_excel_report()` to trigger and confirm the Excel export.
5. Synchronization: Replaced fixed `time.sleep()` delays with `WebDriverWait` and `expected_conditions` for page and menu interactions, and used a folder-polling loop in `download_excel_report()` to detect when the downloaded file finishes writing (ignoring Chrome's temporary `.crdownload` files) so the script does not proceed prematurely.
6. Testing: Ran the script against the live portal, verified that each menu step executed correctly, confirmed that the OPD Register page loaded as expected, and validated that the Excel file was correctly saved to the configured download folder before the browser session closed.

RESULT

The automation script successfully logs into the HMSSO portal, expands the MIS Report and Report menus, opens the OPD Register page, triggers its Excel export, and confirms the file has been fully downloaded into the configured HMSSO_Reports folder, all without manual intervention. This reduced the end-to-end process of reaching the report and obtaining the Excel sheet from several manual clicks to a single script execution.

- (1) Login page before automation
- (2) Terminal output showing 'Logged in.' and menu click confirmations
- (3) OPD Register report page opened by the script
- (4) Terminal output showing 'Downloaded file: ...' with the file path
- (5) The downloaded Excel sheet open in Excel/LibreOffice Calc.]

CONCLUSION

This project provided practical, hands-on experience in web automation using Python and Selenium in a real organizational setting. By automating the login, report-navigation, and Excel-export workflow on CCL's HMSSO hospital portal, routine manual work for the System Department was reduced, and the underlying structure can be extended to automate other MIS reports in the future.

The internship also strengthened understanding of browser automation concepts such as explicit waits, element locators, and driver lifecycle management, which are broadly applicable to test automation and RPA (Robotic Process Automation) tasks beyond this specific project.

REFERENCES

1. Selenium Official Documentation - <https://www.selenium.dev/documentation/>
2. webdriver-manager (Python) - PyPI package documentation
3. Coal India Limited / Central Coalfield Limited - official corporate website
4. Python Software Foundation - official Python documentation